SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Mock Interview

Course Code:	**ITA05**	**Course Title:**	**Computer Vision**
CO Assessed:	**CO5 – Naturalization (independent application using OpenCV)P5**	**Assessment:**	**Assessment Tool 3 – Mock Interview**
Weightage:	**25% of CIA**	**Total Marks:**	**100**
CO4 Statement:	*Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)*		
Student Name:	K.Vijay Bhaskar Reddy	**Reg. No.:**	192425155
Section / Batch:	AI&ML/2024	**Date:**	25/08/2026

Code of Conduct

I K.Vijay Bhaskar Reddy(192425155) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Criteria	Weigh tage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	10	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

34. A system must track detected features across video frames. Design the approach and justify your tracking method.

1. Problem Understanding

The objective is to design a system that can **detect important visual features in a video frame and track the same features across consecutive frames**.

Feature tracking is useful in applications such as:

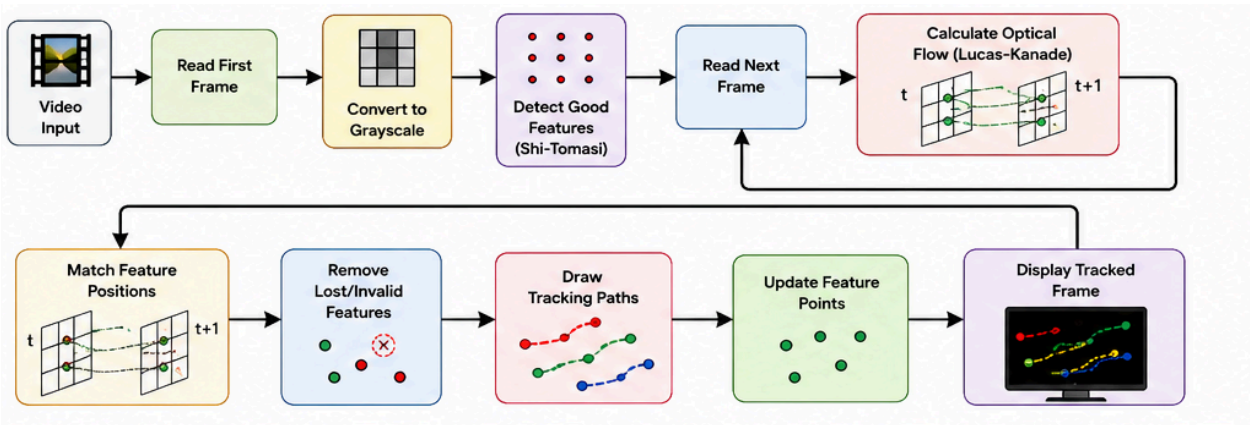
- Object tracking
- Video stabilization
- Motion estimation
- Augmented reality
- Autonomous vehicles
- Surveillance systems
- Structure-from-motion

The system should reliably determine **where each detected feature moves from one frame to the next**.

2. Proposed Approach

A suitable approach is to use **Lucas-Kanade Optical Flow** with **Shi-Tomasi corner detection**.

System Pipeline



3. Feature Detection

First, the system identifies points that can be tracked reliably.

The **Shi-Tomasi Good Features to Track** algorithm is suitable because it identifies strong corners with distinctive intensity changes.

OpenCV function:

`cv2.goodFeaturesToTrack()`

Example:

```
features = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)
```

These detected corners become the initial feature points.

4. Feature Tracking

After detecting features in the first frame, the next frame is captured.

The **Lucas-Kanade Optical Flow** algorithm estimates where each feature has moved.

OpenCV function:

`cv2.calcOpticalFlowPyrLK()`

Example:

```
new_features, status, error = cv2.calcOpticalFlowPyrLK(
    old_gray,
    new_gray,
    features,
    None
)
```

The algorithm compares small neighborhoods around each feature and estimates its displacement between frames.

5. Feature Validation

Not every feature can be tracked successfully. Therefore, the `status` value is checked.

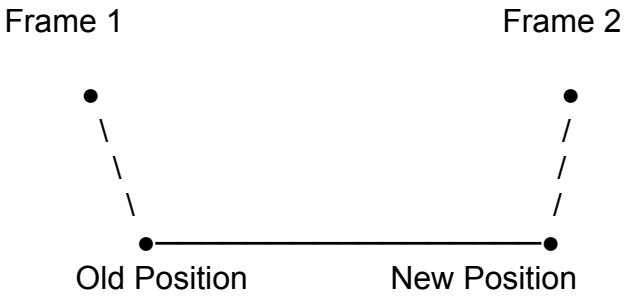
```
good_new = new_features[status == 1]
good_old = features[status == 1]
```

Only successfully tracked features are retained.

This prevents lost points from producing incorrect tracking results.

6. Tracking Path

The movement of each feature can be visualized by drawing a line between its previous and current position.



The accumulated paths provide information about the movement of objects or the camera.

7. Complete Algorithm

```
START

Open video

Read first frame

Convert first frame to grayscale

Detect strong feature points
using Shi-Tomasi

WHILE video contains frames

   Read next frame

   Convert frame to grayscale

   Calculate optical flow
   using Lucas-Kanade

   FOR each feature

      Check tracking status

      IF feature is successfully tracked

         Store old position
```

```
            Store new position

            Draw tracking line
            Draw feature point

        ELSE

            Remove feature

        END IF

    END FOR

    Display tracked frame

    Update previous frame

    Update feature positions

    IF number of features becomes too low

        Detect new features

    END IF

END WHILE

Release video

Close windows

END
```

8. Why Lucas-Kanade Optical Flow?

Lucas-Kanade Optical Flow is appropriate for this problem because it is computationally efficient and specifically designed to estimate the movement of feature points between consecutive frames.

Advantages

Factor	Lucas-Kanade Optical Flow
Computational cost	Low
Real-time tracking	Excellent
Feature-level tracking	Yes

Suitable for video	Yes
OpenCV support	Yes
Small movements	Very effective
Implementation complexity	Low

The **pyramidal implementation** used by OpenCV can also handle larger movements by analyzing the image at multiple resolutions.

9. Handling Tracking Problems

Feature Loss

Features may disappear because of occlusion, blur, or leaving the frame.

Solution: Remove invalid points and detect new features when the number of tracked points becomes too low.

Fast Motion

Large movement between frames can make tracking difficult.

Solution: Use the pyramidal Lucas-Kanade method and, if necessary, increase the video frame rate.

Noise

Noise can cause unstable feature locations.

Solution: Apply Gaussian filtering before feature detection and optical-flow calculation.

Occlusion

A tracked feature may temporarily disappear behind another object.

Solution: Validate tracking status and reacquire features when they become available.

Illumination Changes

Changes in brightness can affect feature matching.

Solution: Use grayscale images and robust feature detection; for severe illumination changes, more advanced feature descriptors can be used.

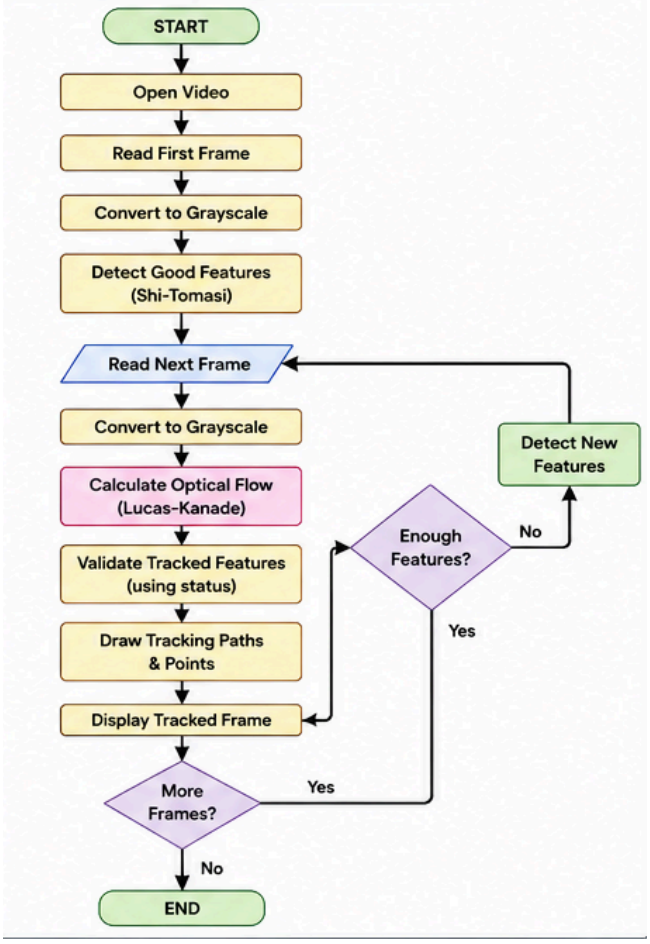
10. Alternative Tracking Methods

Different tracking methods can be selected depending on the application.

Method	Best Use	Advantage
Lucas-Kanade Optical Flow	Feature tracking	Fast and lightweight
KLT Tracker	Corner-feature tracking	Efficient and accurate
CSRT	Object tracking	Robust to scale changes
KCF	Object tracking	Fast
ORB Feature Matching	Feature correspondence	Handles larger transformations
Deep SORT	Multi-object tracking	Strong identity tracking

For this particular problem, **Lucas-Kanade + Shi-Tomasi** is preferred because the requirement specifically concerns **tracking detected features across consecutive video frames**, rather than tracking complete objects.

11. Overall System Diagram



12. Justification

The proposed **Shi-Tomasi + pyramidal Lucas-Kanade Optical Flow** approach provides a good balance between accuracy and computational efficiency. Shi-Tomasi identifies stable corner features, while Lucas-Kanade estimates their displacement between consecutive frames. Invalid features are removed, and new features can be detected when necessary.

Therefore, the approach is suitable for **real-time feature tracking in video**, especially when the features undergo relatively small or moderate movements between consecutive frames. For very large viewpoint changes or long-term feature matching, methods such as **ORB/SIFT-based matching** would be more appropriate.